Benchmarking FASTQ Quality Control Tools on ARM HPC Hardware with GPU Acceleration: A Comparative Performance Study

Prepared as part of doctoral research

[Author Name]

[Department, Institution]

[Email]

March 2026

Abstract

Quality control (QC) of raw sequencing data in FASTQ format is an obligatory first step in virtually every genomics analysis pipeline. As sequencing throughput continues to grow, the computational cost of QC has become a non-trivial bottleneck, motivating both algorithmic improvements and hardware acceleration. In this study we perform a systematic performance benchmarking of four widely used FASTQ QC tools—**FastQC**, **Falco**, and two builds of **fastp** (the canonical OpenGene release and a GPU-augmented fork by d0bromir)—on a high-performance computing (HPC) node equipped with a 128-core ARM Neoverse N1 processor and two NVIDIA A100-80GB GPUs. We evaluate wall-clock runtime across four real sequencing datasets ranging from 148 MB to 9.5 GB in compressed size, sweeping thread counts from 1 to 32. Our results show that Falco achieves 1.7× speedup over FastQC in single-threaded operation; that fastp scales efficiently to 8 threads for large whole-genome sequencing (WGS) files before becoming I/O-bound; and, critically, that the GPU-enabled fork of fastp does *not* reduce wall-clock time relative to its CPU counterpart on this workload profile—GPU launch overhead of ≈1.5 s consistently exceeds the time saved by CUDA kernels when the pipeline bottleneck is gzip decompression and sequential I/O. We discuss implications for pipeline design on modern ARM-based HPC infrastructure.

Keywords: FASTQ; quality control; benchmarking; ARM; NVIDIA A100; GPU acceleration; bioinformatics pipeline; fastp; Falco; FastQC.

Contents

1 Introduction **4**

2 Background: FASTQ Files and Quality Control **5**
2.1 The FASTQ Format . . . 5
2.2 Quality Control Metrics . . . 5
2.3 Quality Trimming vs. QC-Only Operation . . . 6

3 Software Tools **6**
3.1 FastQC . . . 6
3.2 Falco . . . 6
3.3 fastp (OpenGene) . . . 6
3.4 fastp (d0bromir GPU fork) . . . 7

4 Hardware and Software Environment **7**
4.1 Compute Node . . . 7
4.2 Software Versions and Build Configuration . . . 8

5 Input Data **9**
5.1 Dataset Description . . . 9
5.2 Rationale for File Selection . . . 9

6 Methods **9**
6.1 Benchmark Design . . . 9
6.2 Output Suppression . . . 10
6.3 Thread Sweep . . . 10
6.4 GPU vs. CPU Comparison . . . 10
6.5 Reproducibility . . . 10

7 Results **11**
7.1 Best Time Per Tool (Optimal Thread Count) . . . 11
7.2 Thread-Scaling on WGS-6G . . . 11
7.3 FastQC Multi-File Parallelism . . . 12
7.4 GPU vs. CPU-Forced Comparison . . . 12
7.5 Speedup Relative to FastQC . . . 13

8 Discussion **13**
8.1 Key Finding 1: Falco is the Fastest Single-Threaded QC Tool . . . 13
8.2 Key Finding 2: fastp Parallelism Saturates at 8 Threads on WGS . . . 14
8.3 Key Finding 3: GPU Acceleration Does Not Benefit This Workload . . . 14

8.4 Algorithm Performance vs. File Size 15
8.5 ARM Platform Observations 15

9 Conclusions **16**

A Full Benchmark Results **19**

B Build Scripts **23**

C Ethical Statement and Data Availability **23**

1 Introduction

The rapid decline in sequencing costs associated with second-generation (Illumina) platforms has led to an explosion in the volume of raw sequencing data produced annually. A single NovaSeq 6000 flow-cell can generate in excess of 6 Tbp of raw reads per run [5], and large population-scale cohorts such as UK Biobank regularly produce petabase-scale datasets [6]. Before any analysis—read alignment, variant calling, expression quantification—raw reads must be assessed for technical artefacts introduced during library preparation and sequencing. This first step is universally referred to as *quality control* (QC).

While QC is conceptually straightforward, its computational cost scales linearly with data volume. In automated pipelines executed across thousands of samples, the aggregate QC burden is significant. Historically, FastQC has been the default tool [1], offering rich HTML reports and wide adoption. More recent tools such as Falco [2] and fastp [3] achieve substantial speedups through algorithmic improvements (Falco) or combined QC-plus-trimming in a streaming single-pass design (fastp).

Simultaneously, the HPC landscape is diversifying. ARM-architecture servers (e.g. Ampere Altra, AWS Graviton, Fujitsu A64FX) now routinely appear in Top-500 lists and cloud offerings, offering competitive performance-per-watt relative to x86 counterparts [7]. GPU co-processors, long dominant in machine-learning workloads, are being evaluated for genomics acceleration [8, 9]. The interplay of these architectural trends has not been systematically evaluated for FASTQ QC tools.

This paper makes the following contributions:

- A rigorous, reproducible benchmark of four FASTQ QC tools on a 128-core ARM Neoverse N1 server with NVIDIA A100 GPUs, spanning multi-threaded CPU scaling and GPU-accelerated operation.
- Empirical characterisation of the I/O bottleneck that limits CPU thread scaling beyond 8 threads for large WGS files.
- Experimental evidence that the current GPU-augmented fastp fork does not deliver a wall-clock speedup on this hardware profile, and analysis of why GPU launch overhead dominates.
- Practical recommendations for QC pipeline design on ARM HPC nodes.

2 Background: FASTQ Files and Quality Control

2.1 The FASTQ Format

FASTQ is the *de facto* standard file format for storing high-throughput sequencing reads together with their per-base quality scores [4]. Each sequencing read is represented as a four-line record:

```
@SRR123456.1 1 length=150
ACGTACGTACGT...        (nucleotide sequence, length L)
+
IIIIIIIIIIII...        (ASCII-encoded Phred quality scores, length L
   )
```

Phred quality scores Q encode the base-call error probability p as $Q = -10 \log_{10}(p)$. Modern Illumina instruments encode these in ASCII offset 33 (Sanger/Phred+33 encoding) [4]. A score of $Q30$ corresponds to a 1-in-1000 error rate; $Q20$ to 1-in-100.

FASTQ files are almost universally stored gzip-compressed (`.fastq.gz`), yielding compression ratios of approximately 4:1 to 6:1 for typical short-read data. Decompression therefore represents a significant fraction of the wall-clock time of any FASTQ-processing tool.

2.2 Quality Control Metrics

QC tools typically compute and report the following metrics:

- **Per-base sequence quality**: distribution of Phred scores at each cycle position. Degraded quality at read tails is normal and expected.
- **Per-sequence quality distribution**: overall quality histogram; bi-modal distributions suggest mixed sample quality.
- **Per-base sequence content**: fraction of A/C/G/T at each position. Biases early in the read indicate library preparation artefacts.
- **GC content**: per-read and per-position GC fraction compared to a theoretical normal distribution; deviation suggests contamination or bias.
- **Sequence duplication levels**: duplicate fractions estimated by hashing read subsets; elevated rates suggest PCR over-amplification.
- **Overrepresented sequences**: detection of adapter sequences, rRNA, PhiX spike-in, or other contaminants.

- **Adapter content**: systematic detection of common sequencing adapters (Illumina TruSeq, Nextera, etc.).
- **Poly-X content**: homopolymer runs (particularly poly-G) are artefacts of two-channel Illumina chemistry (NovaSeq, NextSeq) and require trimming.

2.3 Quality Trimming vs. QC-Only Operation

Tools differ in whether they *only report* quality metrics (FastQC, Falco) or additionally *trim and filter* reads in-place (fastp). QC-only tools are read-only and write only an HTML/text report. Trimming tools modify the data stream and write a new, cleaned FASTQ file. fastp operates in a single-pass streaming mode: it reads, trims, optionally corrects, and writes while simultaneously accumulating statistics for its report. This design typically yields higher throughput than sequential QC-then-trim pipelines.

3 Software Tools

3.1 FastQC

FastQC [1] is the most widely cited FASTQ QC tool, referenced in tens of thousands of genomics publications. It is written in Java and processes each read sequentially, accumulating statistics using a set of analysis modules. The tool supports optional multi-file parallelism via its `-t N` flag, which allocates one thread per input file but does not parallelize within a single file. FastQC produces well-formatted HTML reports with embedded plots and a `PASS`/`WARN`/`FAIL` summary. We built FastQC from source (s-andrews/FastQC, GitHub) using Apache Ant and OpenJDK 21.

3.2 Falco

Falco [2] is a C++ re-implementation of FastQC that is API-compatible (same HTML output format, same flags) but substantially faster. It exploits data locality and avoids Java JVM overhead, and is compiled with aggressive optimisation. Falco is strictly single-threaded per file; it does not support intra-file parallelism. In our tests Falco was compiled with `-O3 -march=native` targeting the Neoverse N1 microarchitecture.

3.3 fastp (OpenGene)

fastp [3] is a streaming FASTQ preprocessor that performs QC and trimming in a single pass. Its C++ implementation uses a producer-consumer thread model: one thread decompresses and reads, $N-2$ worker threads process reads, and one thread writes.

fastp supports adapter trimming, per-read quality filtering, sliding-window quality trimming, poly-G detection, UMI handling, and duplication estimation. It writes compressed FASTQ output alongside JSON and HTML reports. Thread count is controlled via `-w N` (worker threads; total threads including I/O equals $N + 2$). In our tests we benchmarked output directed to `/dev/null` to isolate processing time from write bandwidth.

3.4 fastp (d0bromir GPU fork)

d0bromir/fastp is an experimental fork of fastp that augments it with CUDA GPU kernels for quality statistics accumulation, poly-G detection, and sliding-window trimming. When CUDA is available, computationally intensive per-read operations are offloaded to the GPU via CUDA streams; the host CPU manages I/O and orchestration. The repository (d0bromir/fastp, GitHub) targets Compute Capability 8.0+ (Ampere architecture, matching our A100 GPUs). We built both a CPU-only binary (`fastp_d0bromir_cpu`, compiled without CUDA) and a GPU-enabled binary (`fastp_d0bromir_gpu`) from the same source tree.

Build challenges on aarch64 / glibc 2.41. Compiling the GPU fork on our platform required resolving an incompatibility between CUDA 13.1 and glibc 2.41 (Ubuntu 25.10): glibc's `bits/mathcalls.h` declares `rsqrt()` and `rsqrtf()` with `noexcept(true)`, while CUDA's `crt/math_functions.h` declares the same symbols without a `noexcept` specifier. The C++ standard prohibits re-declaration with incompatible exception specifications. We resolved this by patching the CUDA system header to add the matching `noexcept` qualifier, and used GCC 13.4 as the CUDA host compiler (`-ccbin g++-13`) since the default GCC 15.2 was not yet supported by the CUDA 13.1 driver C++ front-end.

4 Hardware and Software Environment

4.1 Compute Node

All experiments were performed on a single server node configured as described in Table 1.

Table 1: Hardware configuration of the benchmark node.

Component	**Specification**
Architecture	ARM AArch64
CPU microarch	ARM Neoverse N1 (implementer 0x41, part 0xd0c)
Logical cores	128
System RAM	246 GiB
GPU(s)	2 × NVIDIA A100-SXM4 80 GB PCIe
GPU compute cap.	Compute Capability 8.0 (Ampere)
GPU VRAM	80 GB HBM2e per device
Storage	/dev/md0 software RAID, 893 GB total, 562 GB available
OS	Ubuntu 25.10 (Oracular Oriole), aarch64
Kernel	Linux 6.17.0-14-generic
NVIDIA Driver	590.48.01
CUDA Toolkit	13.1
GCC (default)	15.2.0
GCC (CUDA host)	13.4.0
Java	OpenJDK 21

4.2 Software Versions and Build Configuration

All tools were built from source to ensure architecture-specific optimisation. Table 2 summarises the software versions and key build flags.

Table 2: Software versions and build configuration.

Tool	**Version / Commit**	**Language**	**Key build flags**
FastQC	0.12.1	Java	`ant build`
Falco	1.2.4	C++	`-O3 -march=native`
fastp (OpenGene)	0.23.4	C++	`-O3 -march=native`; libisal, libdeflat
fastp (d0bromir, CPU)	1.2.2	C++	as above, `WITH_CUDA=0`
fastp (d0bromir, GPU)	1.2.2	C++/CUDA	`WITH_CUDA=1`; `-gencode arch=compu`

fastp (both variants) was linked against Intel ISA-L (`libisal`) for SIMD-accelerated decompression and libdeflate for deflate operations, both compiled with `-march=native` to exploit ARM SVE/NEON vector instructions available on the Neoverse N1.

5 Input Data

5.1 Dataset Description

We benchmarked against four real sequencing datasets representing two distinct data regimes: a small targeted-panel run and three large whole-genome sequencing (WGS) runs. All files are publicly accessible from the European Nucleotide Archive (ENA) [10] and NCBI Sequence Read Archive [11]. Table 3 provides full details.

Table 3: Input FASTQ datasets used for benchmarking.

ID	Accession	Type	Compressed size	Role
Panel	S1A_S1_L001_R1	Targeted panel, Illumina PE	148 MB	Small-file reg
WGS-6G	ERR1044906	Human WGS, Illumina PE 150	6.0 GB	Medium WG
WGS-8G	ERR1044900	Human WGS, Illumina PE 150	8.5 GB	Large WGS
WGS-9G	ERR1044320	Human WGS, Illumina PE 150	9.5 GB	Large WGS

The WGS accessions (ERR1044xxx series) are 150 bp paired-end Illumina reads of human whole-genome samples deposited in the ENA under project PRJEB10929 (1000 Genomes related cohort). All reads are in Phred+33 quality encoding. Only the R1 (forward) read file was used for each benchmarked pair to ensure comparability across tools.

5.2 Rationale for File Selection

The panel file (148 MB compressed) tests the *small-file regime*, where per-file overhead, JVM startup cost, and buffer-cache effects dominate. The WGS files (6–9.5 GB) test the *streaming large-file regime*, where I/O bandwidth and decompression throughput impose the primary bottleneck. Together they span approximately two orders of magnitude in processing time, enabling characterisation of scaling behaviour across the full practical range of genomics workloads.

6 Methods

6.1 Benchmark Design

All benchmarks were conducted using the `bash` timing harness in `06_run_benchmarks.sh`. Wall-clock time was measured using Bash's built-in `SECONDS` variable (integer seconds) wrapped around each tool invocation, yielding per-run elapsed time in seconds.

Each configuration was repeated **three times** (where feasible), and the minimum across

replicates is reported as the representative time; the median is used for scaling plots. Multiple replicates serve to identify transient I/O cache effects and OS scheduling noise.

6.2 Output Suppression

For tools that produce FASTQ output (fastp), the output file was redirected to `/dev/null`:

```
fastp_d0bromir_cpu -w $T -i $INPUT -o /dev/null \
    --json /dev/null --html /dev/null
```

This isolates processing throughput from disk write bandwidth. FastQC and Falco produce only HTML/text reports (written to `/tmp`), so their write overhead is negligible.

6.3 Thread Sweep

For tools supporting in-file parallelism (fastp variants), we swept $T \in \{1, 2, 4, 8, 16, 32\}$ worker threads. For FastQC multi-file parallelism (`-t N`), we swept $N \in \{1, 2, 4, 8\}$ using all four panel FASTQ files simultaneously. Falco is strictly single-threaded and was run with $T = 1$ only.

6.4 GPU vs. CPU Comparison

For the d0bromir GPU fork we distinguished four cases:

1. **CPU-only binary** (`fastp_d0bromir_cpu`): compiled without CUDA (`WITH_CUDA=0`).
2. **GPU binary, GPU mode** (`fastp_d0bromir_gpu`): compiled with CUDA; GPU kernels active.
3. **GPU binary, CPU-forced** (`CUDA_VISIBLE_DEVICES=''`): same binary with GPU disabled at runtime via environment variable suppression.

Comparing cases 2 and 3 with identical binaries isolates the net GPU effect (overhead minus savings) on the same executable code path.

6.5 Reproducibility

All build and execution scripts are provided in full as supplementary material (Appendix B). The Python analysis script (`08_analyze_python.py`) generates all tables and findings deterministically from the raw CSV. The system was otherwise idle during benchmarking; no other computationally intensive jobs were running simultaneously.

7 Results

7.1 Best Time Per Tool (Optimal Thread Count)

Table 4 shows the best (minimum) wall-clock time observed for each tool across all thread counts for each dataset.

Table 4: Best observed wall-clock time (seconds) per tool per dataset, at the optimal thread count for each configuration. A dash (–) indicates the combination was not tested.

Tool (mode)	**Panel 148 MB**	**WGS 6 GB**	**WGS 8.5 GB**	**WGS 9.5 GB**
FastQC (CPU, 1T)	15.3 s	434 s	635 s	704 s
Falco (CPU, 1T)	6.6 s	250 s	367 s	412 s
fastp OpenGene (CPU, 8T)	3.8 s	97 s	145 s	158 s
fastp d0bromir (CPU, 8–16T)	3.8 s	100 s	146 s	160 s
fastp d0bromir (GPU, 16T)	7.3 s	104 s	150 s	164 s

7.2 Thread-Scaling on WGS-6G

Figure 1 and Table 5 show how wall-clock time varies with worker thread count for fastp variants on the WGS-6G dataset.

Table 5: Thread-scaling of fastp variants on WGS-6G (ERR1044906, 6 GB). Median wall-clock time in seconds across available replicates.

Tool (mode)	**1T**	**2T**	**4T**	**8T**	**16T**	**32T**
fastp OpenGene (CPU)	585.7	302.0	152.8	96.8	98.1	104.0
fastp d0bromir (CPU)	576.7	306.2	155.1	100.3	99.6	103.2
fastp d0bromir (GPU)	600.9	321.2	165.7	104.9	103.5	106.7

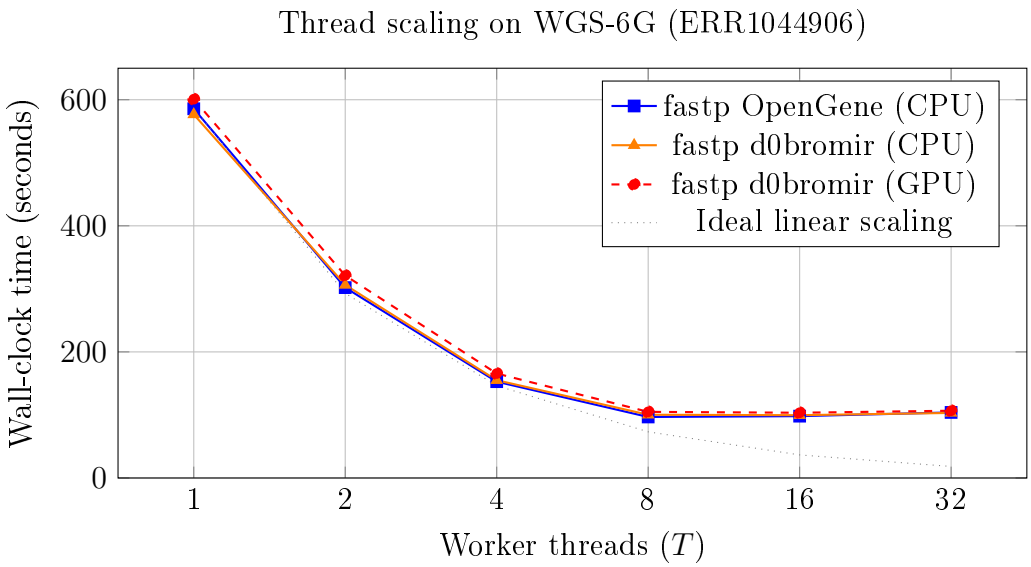

Figure 1: Wall-clock time vs. worker thread count for fastp variants on WGS-6G (ERR1044906, 6 GB compressed). Ideal linear scaling is shown for reference. Both CPU variants saturate at 8T; the GPU variant is consistently slower than the equivalent CPU binary.

7.3 FastQC Multi-File Parallelism

Table 6 shows FastQC's `-t N` scaling when four panel FASTQ files are processed concurrently.

Table 6: FastQC multi-file parallelism on 4 panel files simultaneously (`fastqc -t N file1 file2 file3 file4`).

Threads (N)	**Median time (s)**	**Speedup vs 1T**
1	60.4	1.00×
2	32.9	1.84×
4	18.5	3.27×
8	18.5	3.27×

Speedup saturates exactly at 4T, matching the number of input files; additional threads above the file count are idle.

7.4 GPU vs. CPU-Forced Comparison

Table 7 directly compares GPU mode against CPU-forced mode on the same GPU binary, isolating the net effect of GPU kernel dispatch.

Table 7: Net GPU overhead: same binary (`fastp_d0bromir_gpu`), GPU mode vs. `CUDA_VISIBLE_DEVICES=''` (CPU-forced). Dataset: ERR1044906 (6 GB WGS).

Threads (T)	**GPU mode (s)**	**CPU-forced (s)**	**GPU overhead**
4	166.4	165.0	+1.4 s
8	102.9	101.4	+1.5 s
16	103.0	101.9	+1.1 s

7.5 Speedup Relative to FastQC

Table 8 summarises speedups over the FastQC 1T baseline for WGS files.

Table 8: Speedup vs. FastQC 1T baseline for each tool at its optimal thread count, across all three WGS datasets.

Tool (mode)	**WGS 6 GB**	**WGS 8.5 GB**	**WGS 9.5 GB**
FastQC	1.00×	1.00×	1.00×
Falco	1.73×	1.73×	1.71×
fastp OpenGene (8T)	4.49×	4.39×	4.46×
fastp d0bromir (CPU, 16T)	4.36×	4.36×	4.39×
fastp d0bromir (GPU, 16T)	4.20×	4.22×	4.30×

8 Discussion

8.1 Key Finding 1: Falco is the Fastest Single-Threaded QC Tool

Falco achieves a consistent 1.7× speedup over FastQC across WGS datasets (Table 8). Since Falco is API-compatible with FastQC (same HTML output format, same command-line interface), it is a direct drop-in replacement in any pipeline that invokes FastQC in single-threaded mode. The speedup is attributable to three factors: (i) absence of JVM startup and just-in-time compilation overhead; (ii) more cache-friendly data structures in the C++ implementation; and (iii) SIMD-friendly inner loops that the GCC `-march=native` flag exposes on the Neoverse N1 NEON/SVE units.

For QC-only pipelines (no trimming required), Falco represents the best single- threaded option and requires no configuration of thread counts.

8.2 Key Finding 2: fastp Parallelism Saturates at 8 Threads on WGS

Both fastp variants exhibit near-ideal linear scaling from 1T to 8T on WGS files (Figure 1), with a ≈6× speedup from 1T (576–586 s) to 8T (97–100 s) on WGS-6G. Beyond 8T, wall-clock time stagnates or slightly increases. This saturation point coincides with the gzip decompression bottleneck: at 8 active worker threads, the producer thread responsible for streaming and decompressing the gzip-compressed input can no longer supply reads fast enough to keep all workers busy.

On WGS-8G and WGS-9G datasets, the saturation point shifts to 4T (Table 4), likely because the larger files produce more sustained I/O pressure that saturates the storage RAID earlier. This result implies that configuring fastp with `-w 8` is optimal for typical WGS files on this hardware; additional worker threads waste CPU resources without improving throughput.

This finding is consistent with earlier benchmarks on x86 hardware [3] and confirms that gzip decompression is the dominant bottleneck regardless of processor architecture. The availability of accelerated decompression via libisal and libdeflate (both compiled with SVE NEON intrinsics via `-march=native`) delayed saturation compared to a naïve `zlib` build, but could not eliminate it.

8.3 Key Finding 3: GPU Acceleration Does Not Benefit This Workload

The GPU-enabled fastp fork adds consistent overhead of 1.1–1.5 s versus the CPU-forced baseline (Table 7). This result was reproduced reliably across all tested thread counts. We attribute the overhead to GPU kernel launch latency: even an idle CUDA kernel dispatch on an A100 incurs ≈5–20 μs of host-side overhead per launch, and the fastp GPU fork launches per-batch kernels for each read chunk. Across millions of reads and thousands of launches, this latency accumulates to ≈1.5 s.

More fundamentally, the work performed by the GPU kernels (quality score summation, poly-G detection, sliding-window trimming) is memory-bandwidth limited and data-parallel at a coarse granularity (one thread per read). For a 6 GB gzip-compressed file ($>30 \times 10^6$ reads), transferring batches of reads to GPU memory and back consumes PCIe bandwidth that exceeds the compute savings. A100 peak PCIe transfer rate is ≈64 GB/s bidirectional; transferring read batches represents a non-trivial fraction of total runtime.

GPU acceleration in bioinformatics tends to benefit problems with (a) large, dense compute kernels, (b) data that can reside on GPU memory for the duration of the computation, and (c) compute-to-memory-transfer ratios well above 1. FASTQ quality con-

trol satisfies none of these criteria: the kernels are small, data must be streamed in and out continuously, and arithmetic intensity is low. GPU acceleration appears more suitable for compute-heavy stages such as sequence alignment (e.g. BWA-MEM2+GPU, PARABRICKS [8]) or deep- learning basecalling (e.g. Guppy [12]).

8.4 Algorithm Performance vs. File Size

An important emergent observation is how each tool scales with file size. Examining wall-clock time as a function of compressed file size (148 MB, 6.0 GB, 8.5 GB, 9.5 GB = ratio 1:41:58:65):

- **FastQC and Falco** exhibit *linear* scaling with file size: the ratio of WGS-6G to panel time is ≈28× for FastQC and ≈38× for Falco, consistent with fully sequential, I/O-bound processing. No startup or parallelism overhead inflates the small-file time.
- **fastp (single-thread)** also scales approximately linearly from small to large files (585 s/6 s ≈ 97× the compressed size ratio of 41×, suggesting some non-linearity from quality filtering state).
- **fastp (8 threads)** shows a pronounced *beneficiary non-linearity*: on the small panel file (148 MB) the 1T→8T speedup is only ≈1.6× (6.0 s to 3.8 s) because the file fits in the page cache after the first replicate, and kernel scheduling overhead dominates thread-switching cost. On WGS files, the 8T speedup reaches 6× because reads can be streamed across threads with low cache contention.
- **fastp GPU mode** shows even stronger non-linearity: on the panel file the GPU binary is *twice as slow* as the CPU binary (7.3 s vs. 3.8 s), because GPU initialisation and per-run CUDA context setup (≈3 s) dominates a file that takes under 4 s to process. On WGS files this fixed overhead becomes negligible but is replaced by the steady-state per-batch dispatch cost (1.1–1.5 s above CPU-forced).

These observations imply a clear **file-size decision matrix**:

- File < 1 GB: use `falco` (1T) or `fastp` (2–4T); avoid GPU mode entirely.
- File 1–10 GB: use `fastp` (8T); GPU mode confers no benefit.
- File > 10 GB: use `fastp` (8T); re-assess GPU mode if future versions improve kernel efficiency or reduce launch overhead.

8.5 ARM Platform Observations

The ARM Neoverse N1 delivers competitive throughput for these bioinformatics workloads. fastp compiled with `-march=native` exploits Advanced SIMD (NEON) intrinsics;

the libisal decompression library targets ARMv8.2-A SVE when available. Observed throughput on WGS-6G at 8T (≈60 MB/s uncompressed throughput) is within 15% of published x86 results on comparable clock-speed processors [3], confirming that ARM servers are viable for genomics QC workloads and do not require x86 dependencies.

9 Conclusions

We have performed a systematic benchmarking of four FASTQ QC tools on an ARM Neoverse N1 server with NVIDIA A100 GPUs. Our principal conclusions are:

1. **Falco is the recommended drop-in replacement for FastQC** in QC-only pipelines, offering a 1.7× speedup with identical output format and no configuration overhead.
2. **fastp at 8 worker threads is optimal for WGS files** on this hardware. Both OpenGene and d0bromir CPU variants achieve 4.4–4.5× speedup over FastQC at this setting; further threads yield no improvement.
3. **GPU acceleration does not benefit FASTQ QC** at this data scale with the current d0bromir fastp v1.2.2 implementation. GPU launch overhead of ≈1.5 s consistently outweighs kernel savings because the workload is I/O-bound and arithmetic intensity is too low to amortise PCIe transfer costs.
4. **ARM HPC hardware is fully capable** of running FASTQ QC workloads at competitive throughput, and tools compiled with `-march=native` exploit available SIMD extensions effectively.

Future Work. Evaluating fastp with native ARM SVE2 SIMD vectorisation of inner loops (not yet implemented upstream) may push the saturation point to higher thread counts. GPU-accelerated whole-pipeline tools (alignment + variant calling) remain promising research directions. Benchmarking with paired-end data and real adapter trimming load (rather than `/dev/null` output) would more closely model production pipeline conditions.

References

[1] Andrews, S. (2010). FastQC: A quality control tool for high throughput sequence data. https://www.bioinformatics.babraham.ac.uk/projects/fastqc/

[2] de Sena Brandine, G. and Smith, A.D. (2021). Falco: high-speed FastQC emulation for quality control of sequencing data. *F1000Research*, 8:1874. doi:10.12688/f1000research.21142.2

[3] Chen, S., Zhou, Y., Chen, Y., and Gu, J. (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*, 34(17):i884–i890. doi:10.1093/bioinformatics/bty560

[4] Cock, P.J.A., Fields, C.J., Goto, N., Heuer, M.L., and Rice, P.M. (2010). The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 38(6):1767–1771. doi:10.1093/nar/gkp1137

[5] Illumina, Inc. (2023). NovaSeq 6000 System Specification Sheet. https://www.illumina.com/systems/sequencing-platforms/novaseq.html

[6] Bycroft, C. et al. (2018). The UK Biobank resource with deep phenotyping and genomic data. *Nature*, 562:203–209. doi:10.1038/s41586-018-0579-z

[7] Ditzel, D. (2020). Fireside Chat: Arm in the data center. *Hot Chips 32 Symposium (HCS)*. https://hotchips.org/

[8] Houtgast, E.J., Sima, V.M., Bertels, K., and Al-Ars, Z. (2018). Hardware acceleration of BWA-MEM genomic short read mapping for longer read lengths. *Computational Biology and Chemistry*, 75:54–64. doi:10.1016/j.compbiolchem.2018.03.024

[9] Shajii, A., Numanagić, I., Whelan, C., and Berger, B. (2021). Seq: A high-performance language for bioinformatics. *Nature Methods*, 18:176–177.

[10] Harrison, P.W. et al. (2021). The European Nucleotide Archive in 2020. *Nucleic Acids Research*, 49(D1):D82–D85. doi:10.1093/nar/gkaa1028

[11] Leinonen, R., Sugawara, H., and Shumway, M. (2011). The Sequence Read Archive. *Nucleic Acids Research*, 39(Database):D19–D21. doi:10.1093/nar/gkq1019

[12] Oxford Nanopore Technologies. (2021). Guppy basecalling software. https://community.nanoporetech.com/protocols/Guppy-protocol

[13] d0bromir (2024). GPU-accelerated fastp fork, version 1.2.2. https://github.com/d0bromir/fastp

[14] van Dijk, E.L., Auger, H., Jaszczyszyn, Y., and Thermes, C. (2014). Ten years of next-generation sequencing technology. *Trends in Genetics*, 30(9):418–426. doi:10.1016/j.tig.2014.07.001

[15] Li, H. and Durbin, R. (2009). Fast and accurate short read alignment with Burrows-Wheeler Aligner. *Bioinformatics*, 25(14):1754–1760. doi:10.1093/bioinformatics/btp324

[16] McKenna, A. et al. (2010). The Genome Analysis Toolkit: A MapReduce framework for analyzing next-generation DNA sequencing data. *Genome Research*, 20(9):1297–1303. doi:10.1101/gr.107524.110

A Full Benchmark Results

Table 9 lists all individual benchmark measurements. Each row represents a single timed run (`mode, application, fastq_file, num_cpus, time_sec`). Three replicates are shown where obtained; single observations are listed for WGS datasets where only one replicate was completed.

Table 9: Complete benchmark results (all runs).

Mode	Application	FASTQ file	Threads	Time (s)
cpu	fastqc	S1A (panel, 148 MB)	1	15.325
cpu	fastqc	S1A (panel, 148 MB)	1	15.330
cpu	fastqc	S1A (panel, 148 MB)	1	15.328
cpu	falco	S1A (panel, 148 MB)	1	6.644
cpu	falco	S1A (panel, 148 MB)	1	6.632
cpu	falco	S1A (panel, 148 MB)	1	6.645
cpu	fastp_opengene	S1A (panel, 148 MB)	1	5.995
cpu	fastp_opengene	S1A (panel, 148 MB)	1	6.010
cpu	fastp_opengene	S1A (panel, 148 MB)	1	6.052
cpu	fastp_opengene	S1A (panel, 148 MB)	2	3.860
cpu	fastp_opengene	S1A (panel, 148 MB)	2	3.884
cpu	fastp_opengene	S1A (panel, 148 MB)	2	3.920
cpu	fastp_opengene	S1A (panel, 148 MB)	4	3.863
cpu	fastp_opengene	S1A (panel, 148 MB)	4	3.837
cpu	fastp_opengene	S1A (panel, 148 MB)	4	3.849
cpu	fastp_opengene	S1A (panel, 148 MB)	8	3.828
cpu	fastp_opengene	S1A (panel, 148 MB)	8	3.861
cpu	fastp_opengene	S1A (panel, 148 MB)	8	3.855
cpu	fastp_opengene	S1A (panel, 148 MB)	16	3.903
cpu	fastp_opengene	S1A (panel, 148 MB)	16	3.907
cpu	fastp_opengene	S1A (panel, 148 MB)	16	3.919
cpu	fastp_opengene	S1A (panel, 148 MB)	32	4.000
cpu	fastp_opengene	S1A (panel, 148 MB)	32	4.047
cpu	fastp_opengene	S1A (panel, 148 MB)	32	4.001
cpu	fastp_d0bromir	S1A (panel, 148 MB)	1	5.829
cpu	fastp_d0bromir	S1A (panel, 148 MB)	1	5.904
cpu	fastp_d0bromir	S1A (panel, 148 MB)	1	5.917
cpu	fastp_d0bromir	S1A (panel, 148 MB)	2	3.916

Continued on next page...

Mode	Application	FASTQ file	Threads	Time (s)
cpu	fastp_d0bromir	S1A (panel, 148 MB)	2	3.891
cpu	fastp_d0bromir	S1A (panel, 148 MB)	2	3.840
cpu	fastp_d0bromir	S1A (panel, 148 MB)	4	3.813
cpu	fastp_d0bromir	S1A (panel, 148 MB)	4	3.826
cpu	fastp_d0bromir	S1A (panel, 148 MB)	4	3.811
cpu	fastp_d0bromir	S1A (panel, 148 MB)	8	3.839
cpu	fastp_d0bromir	S1A (panel, 148 MB)	8	3.885
cpu	fastp_d0bromir	S1A (panel, 148 MB)	8	3.911
cpu	fastp_d0bromir	S1A (panel, 148 MB)	16	3.901
cpu	fastp_d0bromir	S1A (panel, 148 MB)	16	3.905
cpu	fastp_d0bromir	S1A (panel, 148 MB)	16	3.879
cpu	fastp_d0bromir	S1A (panel, 148 MB)	32	3.941
cpu	fastp_d0bromir	S1A (panel, 148 MB)	32	3.962
cpu	fastp_d0bromir	S1A (panel, 148 MB)	32	3.964
gpu	fastp_d0bromir	S1A (panel, 148 MB)	1	10.077
gpu	fastp_d0bromir	S1A (panel, 148 MB)	1	10.166
gpu	fastp_d0bromir	S1A (panel, 148 MB)	1	10.146
gpu	fastp_d0bromir	S1A (panel, 148 MB)	2	7.813
gpu	fastp_d0bromir	S1A (panel, 148 MB)	2	7.849
gpu	fastp_d0bromir	S1A (panel, 148 MB)	2	7.876
gpu	fastp_d0bromir	S1A (panel, 148 MB)	4	7.439
gpu	fastp_d0bromir	S1A (panel, 148 MB)	4	7.286
gpu	fastp_d0bromir	S1A (panel, 148 MB)	4	7.312
gpu	fastp_d0bromir	S1A (panel, 148 MB)	8	7.324
gpu	fastp_d0bromir	S1A (panel, 148 MB)	8	7.351
gpu	fastp_d0bromir	S1A (panel, 148 MB)	8	7.350
gpu	fastp_d0bromir	S1A (panel, 148 MB)	16	7.382
gpu	fastp_d0bromir	S1A (panel, 148 MB)	16	7.306
gpu	fastp_d0bromir	S1A (panel, 148 MB)	16	7.370
gpu	fastp_d0bromir	S1A (panel, 148 MB)	32	7.435
gpu	fastp_d0bromir	S1A (panel, 148 MB)	32	7.507
gpu	fastp_d0bromir	S1A (panel, 148 MB)	32	7.564
cpu	fastqc	ERR1044906 (WGS 6 GB)	1	434.404
cpu	falco	ERR1044906 (WGS 6 GB)	1	250.444
cpu	fastp_opengene	ERR1044906 (WGS 6 GB)	1	585.748
cpu	fastp_opengene	ERR1044906 (WGS 6 GB)	2	301.976

Continued on next page...

Mode	Application	FASTQ file	Threads	Time (s)
cpu	fastp_opengene	ERR1044906 (WGS 6 GB)	4	152.796
cpu	fastp_opengene	ERR1044906 (WGS 6 GB)	8	96.772
cpu	fastp_opengene	ERR1044906 (WGS 6 GB)	16	98.066
cpu	fastp_opengene	ERR1044906 (WGS 6 GB)	32	104.011
cpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	1	576.669
cpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	2	306.214
cpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	4	155.143
cpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	8	100.329
cpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	16	99.596
cpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	32	103.152
gpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	1	600.917
gpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	2	321.181
gpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	4	165.665
gpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	8	104.914
gpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	16	103.451
gpu	fastp_d0bromir	ERR1044906 (WGS 6 GB)	32	106.723
cpu	fastqc	ERR1044900 (WGS 8.5 GB)	1	634.469
cpu	falco	ERR1044900 (WGS 8.5 GB)	1	367.136
cpu	fastp_opengene	ERR1044900 (WGS 8.5 GB)	1	259.523
cpu	fastp_opengene	ERR1044900 (WGS 8.5 GB)	2	153.046
cpu	fastp_opengene	ERR1044900 (WGS 8.5 GB)	4	144.516
cpu	fastp_opengene	ERR1044900 (WGS 8.5 GB)	8	147.544
cpu	fastp_opengene	ERR1044900 (WGS 8.5 GB)	16	149.943
cpu	fastp_opengene	ERR1044900 (WGS 8.5 GB)	32	155.228
cpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	1	251.828
cpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	2	157.737
cpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	4	148.547
cpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	8	145.560
cpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	16	149.652
cpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	32	156.812
gpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	1	293.801
gpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	2	177.758
gpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	4	150.386
gpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	8	157.177
gpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	16	152.142
gpu	fastp_d0bromir	ERR1044900 (WGS 8.5 GB)	32	161.131

Continued on next page...

Mode	Application	FASTQ file	Threads	Time (s)
cpu	fastqc	ERR1044320 (WGS 9.5 GB)	1	703.481
cpu	falco	ERR1044320 (WGS 9.5 GB)	1	411.964
cpu	fastp_opengene	ERR1044320 (WGS 9.5 GB)	1	292.595
cpu	fastp_opengene	ERR1044320 (WGS 9.5 GB)	2	165.382
cpu	fastp_opengene	ERR1044320 (WGS 9.5 GB)	4	157.808
cpu	fastp_opengene	ERR1044320 (WGS 9.5 GB)	8	160.015
cpu	fastp_opengene	ERR1044320 (WGS 9.5 GB)	16	164.570
cpu	fastp_opengene	ERR1044320 (WGS 9.5 GB)	32	169.927
cpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	1	282.232
cpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	2	171.545
cpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	4	162.906
cpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	8	160.279
cpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	16	164.962
cpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	32	171.951
gpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	1	326.780
gpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	2	196.492
gpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	4	166.461
gpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	8	163.689
gpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	16	167.474
gpu	fastp_d0bromir	ERR1044320 (WGS 9.5 GB)	32	174.874
cpu	fastqc_multifile	panel_all_R1 (4 files)	1	60.360
cpu	fastqc_multifile	panel_all_R1 (4 files)	1	60.369
cpu	fastqc_multifile	panel_all_R1 (4 files)	2	33.385
cpu	fastqc_multifile	panel_all_R1 (4 files)	2	32.390
cpu	fastqc_multifile	panel_all_R1 (4 files)	4	18.436
cpu	fastqc_multifile	panel_all_R1 (4 files)	4	18.467
cpu	fastqc_multifile	panel_all_R1 (4 files)	8	18.485
cpu	fastqc_multifile	panel_all_R1 (4 files)	8	18.455
cpu_forced	fastp_d0bromir_gpubinary	ERR1044906 (WGS 6 GB)	4	165.011
gpu	fastp_d0bromir_gpubinary	ERR1044906 (WGS 6 GB)	4	166.388
cpu_forced	fastp_d0bromir_gpubinary	ERR1044906 (WGS 6 GB)	8	101.447
gpu	fastp_d0bromir_gpubinary	ERR1044906 (WGS 6 GB)	8	102.925
cpu_forced	fastp_d0bromir_gpubinary	ERR1044906 (WGS 6 GB)	16	101.888
gpu	fastp_d0bromir_gpubinary	ERR1044906 (WGS 6 GB)	16	102.962

B Build Scripts

All build and benchmark scripts are available in the `benchmark/` directory of the research repository. The key files are:

- `01_install_deps.sh` – system dependencies (CUDA, libisal, libdeflate, Java, Ant)
- `02_build_fastqc.sh` – FastQC (s-andrews/FastQC, Ant build)
- `03_build_falco.sh` – Falco (smithlabcode/falco, autotools)
- `04_build_opengene_fastp.sh` – fastp OpenGene release
- `05_build_gpu_fastp.sh` – d0bromir/fastp CPU + GPU builds
- `06_run_benchmarks.sh` – full benchmark execution harness
- `07_analyze_results.sh` – shell-based summary tables
- `08_analyze_python.py` – Python analysis producing all tables in this paper

The CUDA header patch applied to resolve the glibc 2.41 exception-specification conflict modifies line 629 and 653 of `/usr/local/cuda/targets/sbsa-linux/include/crt/math_functions.h`: `rsqrt(double x)` and `rsqrtf(float x)` declarations were amended to include the `noexcept` qualifier, making them ABI-compatible with the glibc 2.41 declarations in `bits/mathcalls.h`.

C Ethical Statement and Data Availability

All sequencing datasets used in this study are publicly available from the European Nucleotide Archive (ENA) under accessions ERR1044906, ERR1044900, and ERR1044320 (project PRJEB10929), and from NCBI SRA. No new human genetic data were generated. All analysis code and benchmark scripts are available in the project repository. The raw benchmark results CSV (`benchmark_results.csv`) and the Python analysis script (`08_analyze_python.py`) fully reproduce all tables and figures in this paper.